Manual del Estudiante — Claude Code Essential

DoIT Consulting · Oct 14, 2026 · @Dante Robles

Bienvenida

En 4 sesiones de 4 horas aprenderás a trabajar con Claude Code como un compañero de desarrollo: planear, implementar, probar, revisar y entregar cambios en un proyecto real.

- **Cómo está organizado:** un capítulo por módulo. Cada uno tiene los conceptos, los comandos que vas a usar y los laboratorios paso a paso.
- **Cómo usarlo:** tenlo abierto durante la clase; los laboratorios dicen qué resultado debes ver al terminar. Al final hay una hoja de referencia rápida y una autoevaluación.
- **Proyecto de práctica:** trabajarás sobre `api-tareas`, una API REST pequeña con pruebas. El instructor te compartirá el repositorio.
- **Versión:** este manual corresponde a Claude Code de octubre de 2026. La herramienta se actualiza seguido; si un comando cambió, `/help` muestra la lista vigente.

Antes de la primera sesión

Termina este checklist al menos un día antes; así la primera hora se dedica a aprender, no a instalar.

- ☐ Cuenta Claude activa (Pro, Max, Team o Enterprise) o acceso por API proporcionado por tu empresa.
- ☐ **Windows:** WSL2 con Ubuntu instalado (`wsl --install` en PowerShell como administrador). **macOS:** terminal y Homebrew.
- ☐ Git configurado: `git config --global user.name "Tu Nombre"` y `git config --global user.email "tu@correo.com"`.
- ☐ GitHub CLI autenticado: `gh auth login`.
- ☐ Node.js 22 o posterior: `node --version`.
- ☐ VS Code o un IDE de JetBrains.
- ☐ En Linux/WSL2: `sudo apt install bubblewrap socat` (los usa el sandbox).
- ☐ Clonar el repositorio `api-tareas` que te enviará el instructor y ejecutar `npm install` y `npm test` (2 pruebas fallan a propósito).

Módulo 1 — Fundamentos

Claude Code es un agente de código: lee tu repositorio, ejecuta comandos, edita archivos y verifica su trabajo. Tú defines el objetivo, apruebas las acciones y revisas el resultado.

Un asistente vs. un agente

	Autocompletado	Agente de código
Alcance	La línea o función que escribes	Tareas completas que tocan

	Autocompletado	Agente de código
		varios archivos
Acciones	Sugiere texto	Lee, ejecuta comandos, edita, corre pruebas
Tu papel	Aceptar o rechazar sugerencias	Dirigir, aprobar y revisar

Modelos disponibles

Alias en `/model`	Modelo	Úsalo para
`fable`	Claude Fable 5.1	Problemas difíciles y razonamiento profundo
`opus`	Claude Opus 5.5	Arquitectura, planeación y cambios complejos
`sonnet`	Claude Sonnet 5.5	Trabajo diario de implementación
`haiku`	Claude Haiku 5.5	Tareas rápidas y mecánicas
`opusplan`	Opus 5.5 + Sonnet 5.5	Opus planea y Sonnet ejecuta

Los modelos actuales admiten ventanas de contexto de hasta 1M tokens en Claude Code. Además del modelo, puedes ajustar el **esfuerzo** con `/effort` (`low`, `medium`, `high`, `xhigh`, `max`, `auto`): más esfuerzo es más razonamiento, más tiempo y más consumo. `/fast` activa fast mode para respuestas más rápidas.

Dónde puedes usarlo: terminal, extensiones de VS Code y JetBrains, app de escritorio, navegador (claude.ai/code) y control remoto desde el móvil.

Lab 1.1 — Instalación

Resultado esperado: `claude doctor` termina sin errores y puedes iniciar sesión.

1. Instala con el método de tu sistema:
   - macOS, Linux o WSL: `curl -fsSL https://claude.ai/install.sh | bash`
   - Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`
   - Alternativa con npm (Node.js 22+): `npm install -g @anthropic-ai/claude-code` (sin `sudo`)
2. Cierra y abre la terminal. Ejecuta `claude --version`.
3. Ejecuta `claude doctor` y corrige lo que reporte.
4. Entra a `api-tareas` y ejecuta `claude`. Inicia sesión con tu cuenta.
5. Pregunta: "Explícame la arquitectura de este proyecto y cómo se ejecutan las pruebas."
6. Escribe `/status` para ver versión, modelo y cuenta. Escribe `/model` para ver el selector.

Módulo 2 — Primeros pasos, contexto y seguridad

La calidad del trabajo de Claude depende de dos cosas que tú controlas: el contexto que le das y los permisos con que trabaja.

CLAUDE.md: la memoria del proyecto

`/init` analiza el repositorio y crea un CLAUDE.md con lo que Claude necesita saber al empezar cada sesión. Existen varios niveles:

Archivo	Alcance	¿Se versiona?
`~/.claude/CLAUDE.md`	Todas tus sesiones	No
`./CLAUDE.md` o `./.claude/CLAUDE.md`	El proyecto, para todo el equipo	Sí
`./CLAUDE.local.md`	Tus preferencias en este proyecto	No (agrégalo a `.gitignore`)

Buenas prácticas: incluye comandos de build y pruebas, convenciones, carpetas que no se tocan y qué significa "terminado". Mantenlo corto y específico. Puedes importar otros archivos con `@ruta/al/archivo.md`. Si tu equipo usa AGENTS.md, escribe `@AGENTS.md` dentro de CLAUDE.md. `/memory` abre los archivos de memoria para editarlos.

Manejo de contexto

- `/context` muestra cuánto contexto está ocupado y por qué.
- `/compact [instrucciones]` resume la conversación para liberar espacio, por ejemplo `/compact enfócate en el módulo de fechas`.
- `/clear` empieza una conversación nueva.
- `/resume` o `claude --continue` retoman una sesión anterior; `/recap` da un resumen de una línea.

Permisos y seguridad

Shift+Tab cambia el modo de permisos:

Modo	Qué hace
Manual (`default`)	Pide aprobación antes de editar o ejecutar
`acceptEdits`	Acepta ediciones de archivos; sigue preguntando por comandos
`plan`	Solo explora y propone un plan; no edita
`auto`	Aprueba acciones según un clasificador de seguridad (si tu organización lo permite)

Las reglas en `.claude/settings.json` afinan qué se permite. `deny` siempre gana sobre `allow`:

```
{
  "permissions": {
    "allow": ["Bash(npm run *)"],
    "ask": ["Bash(git push *)"],
    "deny": ["Read(./.env)"]
  }
}
```

`/sandbox` aísla el sistema de archivos y la red de los comandos que Claude ejecuta (macOS, Linux y WSL2).

Deshacer cambios: checkpoints

Claude guarda un punto de control antes de cada petición. Pulsa Esc dos veces con el prompt vacío, o escribe `/rewind`, para restaurar código, conversación o ambos. Importante: no deshace efectos de comandos Bash como `rm` o `mv`. Para historial permanente, usa Git.

Lab 1.2 — Tu primer CLAUDE.md

Resultado esperado: un CLAUDE.md de 20 a 40 líneas que Claude respeta.

1. En `api-tareas`, ejecuta `/init`.
2. Lee el resultado y elimina lo genérico.
3. Agrega: comando de pruebas, una convención de nombres, una carpeta que no debe modificarse y la regla "una tarea está terminada cuando todas las pruebas pasan".
4. Ejecuta `/context` y observa el espacio que ocupa CLAUDE.md.
5. `/clear` y pide un cambio pequeño. Comprueba que Claude corre las pruebas por su cuenta.

Lab 1.3 — Permisos y rewind

Resultado esperado: `.env` inaccesible y un cambio revertido.

1. Crea `.claude/settings.json` con las reglas del ejemplo de arriba.
2. Pide a Claude: "Muéstrame el contenido de .env". Observa el bloqueo.
3. Usa Shift+Tab para pasar a `acceptEdits` y pide renombrar una función.
4. Pulsa Esc dos veces y restaura solo el código.
5. Comprueba con `git status` que no quedaron cambios.

Módulo 3 — Integraciones y extensibilidad

Claude Code se adapta a tu equipo con cuatro piezas: MCP para conectar herramientas, hooks para automatizar reglas, skills para enseñar procedimientos y plugins para empaquetar y compartir todo lo anterior.

Pieza	Qué es	Dónde vive	Cuándo actúa
MCP	Conexión a herramientas y datos externos	`.mcp.json` o `~/.claude.json`	Cuando Claude decide usar la herramienta
Hook	Comando que corre en	`settings.json`	Siempre, de forma

Pieza	Qué es	Dónde vive	Cuándo actúa
	un evento		determinista
Skill	Instrucciones y archivos de un procedimiento	`.claude/skills/<nombre>/SKILL.md`	Cuando la petición coincide con su descripción, o con `/nombre`
Plugin	Paquete de skills, hooks, agentes y MCP	Marketplaces vía `/plugin`	Al instalarlo, según lo que contenga

MCP

```
claude mcp add --transport http <nombre> <url>
claude mcp add --transport stdio <nombre> -- <comando> [args]
```

Las opciones van antes del nombre. `--scope` define el alcance: `local` (por defecto, solo tú en este proyecto), `project` (en `.mcp.json`, compartido con el equipo) o `user` (tú, en todos tus proyectos). Revisa el estado con `claude mcp list` o con `/mcp` dentro de la sesión.

Hooks

Eventos principales: `PreToolUse` (antes de usar una herramienta), `PostToolUse` (después), `UserPromptSubmit`, `Stop` y `SessionStart`. El `matcher` elige herramientas, por ejemplo `Edit|Write` o `Bash`. Si un hook `PreToolUse` termina con código de salida 2, la acción se bloquea y Claude recibe el mensaje de stderr.

Skills

Un SKILL.md mínimo:

```
---
name: nuevo-endpoint
description: Agrega un endpoint REST a api-tareas con su validación y su prueba en
Vitest
---
1. Crea la ruta en src/routes siguiendo el patrón existente.
2. Valida la entrada con el esquema del módulo.
3. Escribe la prueba en tests/ y ejecuta npm test.
```

La `description` decide cuándo se activa: escríbela con las palabras que usarías al pedir la tarea.

Lab 2.1 — Conectar servidores MCP

Resultado esperado: `/mcp` muestra ambos servidores conectados.

1. Agrega Context7 al proyecto: `claude mcp add --scope project --transport http context7 https://mcp.context7.com/mcp`
2. Agrega Playwright solo para ti: `claude mcp add --transport stdio playwright -- npx @playwright/mcp@latest`
3. Abre `.mcp.json` y observa qué se guardó.
4. Pide: "Consulta en Context7 la documentación actual de Vitest y agrega una prueba para fechas inválidas."

Lab 2.2 — Hooks

Resultado esperado: archivos formateados automáticamente y un comando peligroso bloqueado.

1. Crea `.claude/hooks/format.sh`, dale permiso con `chmod +x`, y agrega en `.claude/settings.json` un hook `PostToolUse` con matcher `Edit|Write` que lo ejecute. El script lee el archivo editado con `jq -r '.tool_input.file_path'` y corre `npx prettier --write` sobre él.
2. Pide un cambio con mal formato y verifica que quede formateado.
3. Agrega un hook `PreToolUse` con matcher `Bash` que termine con `exit 2` si el comando contiene `rm -rf`.
4. Pide a Claude que borre la carpeta `dist` con `rm -rf` y observa cómo reacciona al bloqueo.

Lab 2.3 — Skill y plugin

Resultado esperado: una skill que se activa sola y un plugin instalado.

1. Crea la skill `nuevo-endpoint` del ejemplo.
2. Pide: "Agrega un endpoint para marcar una tarea como completada." Verifica que Claude siga los pasos de la skill.
3. Escribe `/nuevo-endpoint` para invocarla directamente.
4. Abre `/plugin`, explora la pestaña Discover e instala `commit-commands@claude-plugins-official`.

Módulo 4 — Flujo de desarrollo asistido

El flujo seguro tiene seis pasos y siempre empieza en una rama: rama → prueba que reproduce → cambio → pruebas en verde → revisión → commit.

1. `git switch -c fix/descripcion` (o pídeselo a Claude).
2. Pide una prueba que reproduzca el problema y confirma que falla.
3. Pide la corrección sin modificar la prueba.
4. Ejecuta todas las pruebas; con `!npm test` las corres tú mismo desde el prompt.
5. Revisa: `/diff` para ver los cambios y `/code-review` para una revisión automática.
6. Pide el commit con un mensaje convencional y revisa el mensaje antes de aceptarlo.

Iniciar un proyecto nuevo: pide primero 2 o 3 alternativas de stack con pros y contras, decide tú, y registra la decisión y los comandos en CLAUDE.md antes de generar código.

Revisión de código

- `/code-review` (alias `/review`): revisa los cambios. Acepta un nivel (`low` a `max`), `--fix` para aplicar correcciones y `--comment` para publicar los hallazgos en el PR.
- `/security-review`: revisa tu rama contra la rama principal de `origin` buscando vulnerabilidades.
- Cada hallazgo que se repite es una regla que falta en CLAUDE.md.

Estilos de salida: `/output-style` cambia cómo responde Claude. `Learning` explica y te deja partes por escribir; `Explanatory` agrega contexto educativo; `Concise` va al grano.

Lab 3.1 — Proyecto nuevo

Resultado esperado: proyecto que compila con al menos una prueba en verde.

1. Crea una carpeta vacía y ejecuta `claude`.
2. Pide: "Propón 2 stacks para una API de inventario con autenticación; compara pros y contras."
3. Elige uno y pide el esqueleto con pruebas y README.
4. Ejecuta `/init` y agrega el stack y los comandos a CLAUDE.md.

Lab 3.2 — Corregir un bug con pruebas

Resultado esperado: una rama con 2 commits y todas las pruebas en verde.

1. En `api-tareas`, lee el issue #3 que preparó el instructor.
2. Sigue los 6 pasos del flujo seguro sin ayuda.
3. Revisa el diff línea por línea antes de cada commit.

Lab 3.3 — Revisión y mejora de CLAUDE.md

Resultado esperado: hallazgos atendidos y al menos 2 reglas nuevas en CLAUDE.md.

1. Ejecuta `/code-review` sobre tu rama y atiende los hallazgos.
2. Ejecuta `/security-review` (requiere el remoto `origin`).
3. Agrega a CLAUDE.md una regla por cada hallazgo que no quieras volver a ver.
4. Prueba `/output-style Learning` en una tarea pequeña y compara con el estilo por defecto.

Módulo 5 — Planeación, ejecución y operación en equipo

Para tareas de más de unos minutos, planea primero: un buen plan aprobado ahorra más tiempo que cualquier atajo durante la implementación.

Plan Mode

En Plan Mode Claude lee, investiga y propone un plan, pero no edita archivos hasta que lo apruebas. Tres formas de entrar:

- Shift+Tab hasta ver `plan` (desde Manual, dos pulsaciones).
- `/plan <descripción de la tarea>`.
- `claude --permission-mode plan` al iniciar.

Revisa el plan como revisarías el de un colega: ¿falta algo?, ¿toca archivos que no debe?, ¿cómo se va a verificar? Pide ajustes antes de aprobar.

Elegir modelo y esfuerzo

- `/model opusplan`: Opus 5.5 planea y Sonnet 5.5 implementa. Buen punto de partida.
- `/effort high` para tareas complejas; `low` para cambios mecánicos.
- `fallbackModel` en settings define hasta 3 modelos de respaldo si el principal no está disponible.

Consumo

`/usage` (también `/cost` o `/stats`) muestra el uso de la sesión y los límites de tu plan. El monto en dólares es una estimación. Para ahorrar: `/clear` entre tareas distintas, `/compact` en sesiones largas, y Haiku o esfuerzo bajo para tareas simples.

En equipo: tu organización puede fijar políticas (modelos permitidos, servidores MCP aprobados, Auto mode activo o no). Si una opción no aparece, probablemente es una política, no un error.

Siguiente nivel: los subagents (`.claude/agents/*.md`) y los worktrees permiten trabajar en paralelo. Se ven a fondo en el curso Multi-Agent + OpenSpec.

Mini-proyecto final

Entregable: un Pull Request en el que aplicas todo el curso. Tienes 110 minutos y 3 minutos para presentarlo.

1. Elige uno de los 3 issues preparados o una mejora de tu propio proyecto.
2. `/model opusplan` y `/plan <tu tarea>`. Ajusta el plan y apruébalo. Guarda el texto del plan.
3. Crea la rama e implementa siguiendo el flujo seguro del Módulo 4.
4. Usa al menos un hook y una skill (pueden ser los de la Sesión 2).
5. Actualiza CLAUDE.md con lo aprendido.
6. `/code-review` y atiende los hallazgos.
7. Pide: "Crea un PR con gh con resumen, cambios y cómo se probó." Revisa la descripción antes de publicarla.
8. Prepara tu demo: problema, plan, resultado y una lección aprendida.

Cómo se evalúa (50 puntos): plan (10), contexto en CLAUDE.md (10), hook y skill funcionando (10), pruebas y revisión atendida (10), PR y demo (10).

Hoja de referencia rápida

Comandos dentro de la sesión

Comando	Para qué
`/help`	Lista de comandos disponibles
`/init`	Crear o mejorar CLAUDE.md
`/memory`	Editar archivos de memoria
`/context`	Ver ocupación del contexto
`/compact [instrucciones]`	Resumir la conversación
`/clear`	Conversación nueva
`/resume`	Retomar una sesión anterior
`/recap`	Resumen de una línea de la sesión
`/model`	Cambiar de modelo

Comando	Para qué
`/effort`	Ajustar el nivel de esfuerzo
`/fast`	Activar o desactivar fast mode
`/plan`	Entrar a Plan Mode con una tarea
`/permissions`	Ver y editar reglas de permisos
`/sandbox`	Configurar el sandbox
`/rewind`	Restaurar un checkpoint
`/mcp`	Estado de servidores MCP
`/hooks`	Ver y editar hooks
`/plugin`	Administrar plugins y marketplaces
`/diff`	Ver cambios sin confirmar
`/code-review` (`/review`)	Revisión de código
`/security-review`	Revisión de seguridad
`/output-style`	Cambiar estilo de respuesta
`/usage` (`/cost`)	Consumo y límites
`/status`	Versión, modelo y cuenta
`/doctor`	Diagnóstico dentro de la sesión
`/config`	Configuración general

Atajos de teclado

Atajo	Acción
Shift+Tab	Cambiar modo de permisos
Esc	Interrumpir a Claude
Esc Esc (prompt vacío)	Abrir el menú de rewind
@	Mencionar un archivo
`!` al inicio	Ejecutar un comando de shell
Ctrl+R	Buscar en el historial
Ctrl+O	Ver el transcript completo
Option+P / Alt+P	Cambiar de modelo

Línea de comandos

Comando	Para qué
`claude`	Iniciar sesión interactiva
`claude -c`	Continuar la última sesión
`claude -p "tarea"`	Modo no interactivo (scripts)
`claude --model opus`	Iniciar con un modelo
`claude --permission-mode plan`	Iniciar en Plan Mode
`claude mcp add / list / remove`	Administrar servidores MCP
`claude doctor`	Diagnóstico de instalación
`claude update`	Actualizar Claude Code

Glosario

Término	Definición
Agente de código	Programa con un modelo de IA que planea y ejecuta acciones sobre tu código, no solo sugiere texto
Contexto	Todo lo que el modelo tiene presente en una conversación: mensajes, archivos leídos, CLAUDE.md y resultados de herramientas
Token	Unidad en que el modelo mide texto; el contexto y el consumo se cuentan en tokens
CLAUDE.md	Archivo de instrucciones que Claude carga al iniciar cada sesión
AGENTS.md	Archivo de instrucciones estándar entre herramientas de IA; se puede importar desde CLAUDE.md
Compactar	Resumir la conversación para liberar contexto
Checkpoint	Punto de restauración que Claude guarda antes de cada petición
Modo de permisos	Nivel de autonomía de Claude: Manual, acceptEdits, plan o auto
Sandbox	Aislamiento del sistema de archivos y la red para

Término	Definición
	los comandos que ejecuta Claude
MCP	Model Context Protocol: estándar para conectar herramientas y datos externos
Hook	Comando que se ejecuta automáticamente en un evento de Claude Code
Skill	Procedimiento empaquetado en un SKILL.md que Claude carga cuando es relevante
Plugin	Paquete instalable de skills, hooks, agentes y servidores MCP
Marketplace	Catálogo de plugins que se agrega con `/plugin marketplace add`
Plan Mode	Modo en que Claude solo explora y propone un plan sin editar
Esfuerzo (effort)	Cuánto razona el modelo antes de responder
Subagent	Agente secundario con su propio contexto al que Claude delega una tarea
Worktree	Copia de trabajo adicional de un repo Git, útil para trabajo en paralelo

Autoevaluación

Responde sin consultar el manual; las respuestas están en el capítulo de cada módulo.

Módulo 1

1. ¿Qué diferencia a un agente de código de un autocompletado?
2. ¿Qué alias de modelo usa Opus para planear y Sonnet para implementar?
3. ¿Qué comando revisa tu instalación sin abrir una sesión?

Módulo 2

4. ¿Qué archivo usas para preferencias personales que no deben subirse al repositorio?
5. Si una regla `allow` y una `deny` coinciden con la misma acción, ¿cuál gana?
6. ¿Por qué `/rewind` no sustituye a Git?

Módulo 3

7. ¿Qué alcance de MCP guarda la configuración en `.mcp.json`?
8. ¿Con qué código de salida un hook `PreToolUse` bloquea una acción?
9. ¿Qué campo de SKILL.md decide cuándo se activa una skill?

Módulo 4

10. ¿Por qué conviene escribir primero la prueba que reproduce el bug?
11. ¿Qué necesita el repositorio para que funcione `/security-review`?

Módulo 5

12. ¿Qué hace Claude en Plan Mode y qué no hace?
13. Menciona dos acciones para reducir el consumo de tokens.

Respuestas: 1) el agente ejecuta y verifica acciones completas · 2) `opusplan` · 3) `claude doctor` · 4) `CLAUDE.local.md` · 5) `deny` · 6) no revierte efectos de comandos Bash ni da historial permanente · 7) `project` · 8) 2 · 9) `description` · 10) para verificar que la corrección realmente resuelve el problema · 11) un remoto `origin` · 12) explora y propone un plan; no edita archivos hasta que lo apruebas · 13) `/clear` entre tareas, `/compact`, modelos o esfuerzo menores para tareas simples.